

Veille Technique - Spark Structured Streaming pour l'IoT

SmartTech - Traitement de Données en Temps Réel

1. Introduction au Streaming avec Spark

1.1 Contexte SmartTech

SmartTech gère des flux massifs de données provenant de capteurs IoT installés dans des bâtiments intelligents. Ces données doivent être traitées en temps réel pour :

- Détecter rapidement des comportements anormaux
- Alimenter des tableaux de bord en temps réel
- Historiser les données pour l'analyse

1.2 Principe du Streaming Structuré

Spark Structured Streaming traite les flux de données comme des **tables infinies** :

- Chaque nouvel enregistrement est une ligne ajoutée à la table
- Les requêtes s'exécutent de manière incrémentale
- API unifiée pour le batch et le streaming
- Garanties de cohérence exactement comme en SQL

Avantages :

- Programmation déclarative (SQL-like)
 - Tolérance aux pannes automatique
 - Optimisations Catalyst et Tungsten
 - Intégration native avec l'écosystème Spark
-

2. Architecture et Concepts Fondamentaux

2.1 Lecture et Écriture de Flux

Sources Supportées

Source Description		Use Case SmartTech
File	JSON, CSV, Parquet, ORC	Fichiers de logs des capteurs

Source	Description	Use Case SmartTech
Kafka	Message broker distribué	Flux temps réel des capteurs
Socket	Connexion TCP	Tests et prototypage
Rate	Génération de données test	Benchmarking

Sinks Supportés

Sink	Description	Use Case SmartTech
Delta Lake	Format ACID avec versioning	Stockage Bronze/Silver/Gold
Kafka	Republication des résultats	Communication inter-services
Console	Affichage debug	Développement et tests
Memory	Table en mémoire	Tests unitaires
Foreach	Traitement personnalisé	Envoi d'alertes

2.2 Modes de Sortie

Append (Ajout)

- Seules les **nouvelles lignes** sont écrites dans le sink
- Lignes jamais modifiées une fois écrites
- **Use Case SmartTech** : Données brutes des capteurs (Bronze)
- **Performance** : Optimal, pas de réécriture

```
python query =
stream.writeStream.outputMode("append").format("delta").start("/
path/to/bronze")
```

Update (Mise à jour)

- Seules les lignes **modifiées** sont réécrites
- Supporte les agrégations avec état
- **Use Case SmartTech** : Compteurs par capteur, moyennes glissantes
- **Performance** : Modéré, écritures sélectives

```
python query =
(stream.groupBy("sensor_id").count() .writeStream .outputMode("update") .f
path/to/aggregates"))
```

Complete (Complet)

- **Toute la table résultat** est réécrite à chaque micro-batch
- Uniquement pour les agrégations
- **Use Case SmartTech** : Dashboards nécessitant la vue complète
- **Performance** : Coûteux, à éviter pour grandes tables

```
python query =
(stream.groupBy("building_id").sum("energy") .writeStream .outputMode("com
```

3. Tolérance aux Pannes et Checkpointing

3.1 Principe du Checkpointing

Le checkpointing sauvegarde l'état du streaming pour permettre une reprise exacte après une panne :

- **Offsets des sources** : Position de lecture (ex: offsets Kafka)
- **État des agrégations** : Compteurs, sommes, fenêtres en cours
- **Métadonnées** : Configuration du query

3.2 Configuration

```
python query = (stream.writeStream .option("checkpointLocation",  
"/path/to/checkpoint") .start())
```

3.3 Garanties ACID avec Delta Lake

La combinaison **Checkpointing + Delta Lake** garantit :

- **Exactly-once** : Chaque enregistrement traité une et une seule fois
- **Atomicité** : Transactions complètes ou annulées
- **Idempotence** : Rejeu sécurisé après panne

Cas SmartTech : En cas de panne du serveur Spark, le pipeline reprend automatiquement au dernier offset traité, sans perte ni doublon de données.

4. Triggers - Contrôle du Traitement

4.1 Types de Triggers

ProcessingTime

- Traitement à **intervalles réguliers**
- **Use Case SmartTech** : Traitement toutes les 5-10 secondes
python .trigger(processingTime="10 seconds")

Once

- **Une seule** exécution puis arrêt
- **Use Case SmartTech** : Jobs batch schedulés (ex: agrégation nocturne)
python .trigger(once=True)

Continuous

- Mode **ultra-faible latence** (< 1 ms)
- Limité en fonctionnalités (pas d'agrégations complexes)

- **Use Case SmartTech** : Détection d'anomalies critiques
python .trigger(continuous="1 second")

AvailableNow

- Traite **toutes les données disponibles** puis s'arrête
- **Use Case SmartTech** : Rattrapage après maintenance
python .trigger(availableNow=True)

4.2 Recommandations SmartTech

Scénario	Trigger	Justification
Données normales	ProcessingTime (10s)	Équilibre latence/throughput
Alertes critiques	Continuous (1s)	Latence minimale
Agrégations nocturnes	Once	Économie de ressources
Après maintenance	AvailableNow	Rattrapage rapide

5. Fenêtres Temporelles et Watermarks

5.1 Windowing (Fenêtrage)

Agrégation des données sur des **intervalles de temps** :

Tumbling Windows (Fenêtres Fixes)

- Pas de chevauchement
- **Use Case SmartTech** : Consommation énergétique par heure
```python from pyspark.sql.functions import window

```
stream.groupBy(window("timestamp", "1 hour"),
"buildingid").sum("energyconsumption") ```
```

### Sliding Windows (Fenêtres Glissantes)

- Chevauchement configurable
- **Use Case SmartTech** : Température moyenne sur 10 min, mise à jour chaque minute  
python result =  
stream.groupBy( window("timestamp", "10 minutes", "1  
minute"), "sensor\_id" ).avg("temperature")

## 5.2 Watermarks - Gestion des Données Tardives

Les watermarks définissent **combien de temps attendre** les données en retard :

```
python result = (stream.withWatermark("timestamp", "15
minutes") .groupBy(window("timestamp", "5 minutes"),
"building_id") .count())
```

## Fonctionnement :

1. Watermark = MAX(event\_time) - 15 minutes
2. Les données avec event\_time < watermark sont ignorées
3. L'état des fenêtres anciennes est libéré

**Cas SmartTech** : Un capteur envoie une mesure avec 20 minutes de retard (problème réseau). Avec un watermark de 15 minutes, cette donnée sera **ignorée** pour ne pas conserver indéfiniment l'état en mémoire.

## 5.3 Trade-off Watermark

| Watermark      | Avantages                | Inconvénients              |
|----------------|--------------------------|----------------------------|
| Court (5 min)  | Mémoire optimale         | Perte de données tardives  |
| Long (1 heure) | Capture données tardives | Forte consommation mémoire |

**Recommandation SmartTech** : 15-30 minutes (compromis pour réseau IoT)

---

# 6. Architecture Médaillon

## 6.1 Principe

Architecture en **trois couches** pour transformation progressive des données :

[Sources] → [Bronze] → [Silver] → [Gold] → [Analytics/BI]

## 6.2 Couche Bronze (Raw/Brute)

**Objectif** : Ingestion brute des données sources

### Caractéristiques :

- Données **immuables** (append-only)
- Schéma minimal ou permissif
- Conservation de TOUTES les données (même invalides)
- Métadonnées d'ingestion (timestamp, source, etc.)

### Transformations SmartTech :

- Parsing du JSON Kafka
- Ajout de ingestion\_time
- Filtrage des nulls critiques uniquement
- Écriture en Delta Lake

```
python bronze =
(raw_stream .filter(col("sensor_id").isNotNull()) .withColumn("ingestion_t"
current_timestamp()) .writeStream .format("delta") .outputMode("append")
bronze/sensors"))
```

## 6.3 Couche Silver (Cleaned/Enriched)

**Objectif :** Données nettoyées et normalisées

**Caractéristiques :**

- Schéma structuré et validé
- Dédoublonnage et validation qualité
- Enrichissement avec données de référence
- Typages corrects et conversions

**Transformations SmartTech :**

- Filtrage des valeurs aberrantes (température hors plage)
- Calculs dérivés (confort thermique, qualité air)
- Normalisation des formats
- Jointures avec tables de référence (bâtiments, capteurs)

```
python silver =
(bronze_stream .filter(col("temperature").between(-50,
60)) .withColumn("comfort_index",
when((col("temp").between(20,24)) &
(col("humidity").between(40,60)),
"comfortable") .otherwise("uncomfortable")) .writeStream .format("delta")
silver/sensors"))
```

## 6.4 Couche Gold (Aggregated/Business)

**Objectif :** Agrégations métier prêtes pour l'analyse

**Caractéristiques :**

- Agrégations par dimension métier
- Calculs KPI (consommation totale, anomalies/jour)
- Dénormalisation pour performance BI
- Mises à jour incrémentales

**Transformations SmartTech :**

- Consommation énergétique par bâtiment/jour
- Taux d'anomalies par zone
- Température moyenne par étage
- Tableaux de bord temps réel

```
python gold = (silver_stream .groupBy(window("timestamp", "1
day"),
"building_id") .agg(sum("energy_consumption").alias("daily_energy"),
avg("temperature").alias("avg_temp"), sum(when(col("anomaly"),
1)).alias("anomaly_count")) .writeStream .format("delta") .outputMode("upd
gold/daily_stats"))
```

## 6.5 Avantages pour SmartTech

| Avantage        | Description                                                              |
|-----------------|--------------------------------------------------------------------------|
| Traçabilité     | Données brutes toujours disponibles (Bronze)                             |
| Réutilisabilité | Silver partagée entre plusieurs équipes                                  |
| Performance     | Gold optimisée pour BI (pré-agrégée)                                     |
| Évolutivité     | Ajout de nouvelles transformations sans impacter les couches précédentes |
| Qualité         | Validation progressive à chaque étape                                    |

---

## 7. Apache Kafka - Message Broker

### 7.1 Rôle dans une Architecture Temps Réel

#### Découplage Producteurs/Consommateurs

- Les capteurs (producteurs) et Spark (consommateur) sont **indépendants**
- Ajout/retrait de consommateurs sans impact sur les producteurs
- Plusieurs consommateurs peuvent lire le même flux

#### Buffer et Absorption des Pics

- Kafka **persiste** les messages (durée configurable : heures/jours)
- Absorption des pics de charge sans perte de données
- Les consommateurs traitent à leur rythme

#### Haute Disponibilité

- Réplication des messages sur plusieurs brokers
- Tolérance aux pannes de serveurs
- Pas de point unique de défaillance

### 7.2 Concepts Clés Kafka

#### Offsets

**Définition** : Position **unique** d'un message dans une partition

#### Caractéristiques :

- Séquence monotone : 0, 1, 2, 3, ...
- Unique par partition
- Immuable

#### Utilité SmartTech :

- Reprise exacte après panne (Spark lit depuis le dernier offset traité)

- Garantie exactly-once avec checkpointing
- Audit et traçabilité

Partition 0: [msg0] [msg1] [msg2] [msg3] [msg4] Offsets: 0 1 2 3  
4 ↑ Consumer Position

## Partitions

**Définition** : Division **logique** d'un topic pour parallélisme

**Caractéristiques** :

- Ordre garanti **au niveau partition** (pas entre partitions)
- Distribution basée sur clé (ex: sensor\_id) ou round-robin
- Immuables après écriture

**Scalabilité SmartTech** :

- Topic `iot_sensors` avec 10 partitions
- 10 tasks Spark lisent en parallèle
- Throughput multiplié par 10

```
Topic: iot_sensors |— Partition 0: [sensor_001, sensor_011,
sensor_021, ...] |— Partition 1: [sensor_002, sensor_012,
sensor_022, ...] |— Partition 2: [sensor_003, sensor_013,
sensor_023, ...] |— ...
```

**Choix du nombre de partitions** :

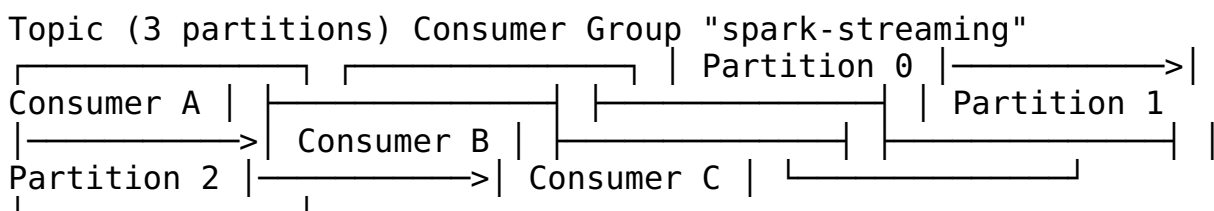
- # Partitions  $\geq$  # Consommateurs souhaités
- SmartTech : 3-10 partitions pour ~50 capteurs

## Consumer Groups

**Définition** : Groupe de consommateurs **partageant** la lecture d'un topic

**Fonctionnement** :

- Chaque partition est lue par **un seul** consommateur du groupe
- Rééquilibrage automatique si consommateur tombe
- Plusieurs groupes peuvent lire le même topic



**Cas SmartTech** :

- Consumer Group "spark-bronze" : Ingestion Bronze
- Consumer Group "spark-alerts" : Détection d'anomalies



- Consumer Group "archival" : Sauvegarde S3

Les trois groupes lisent **indépendamment** le même flux.

## 7.3 Intégration Spark + Kafka

```
python kafka_stream =
(spark.readStream .format("kafka") .option("kafka.bootstrap.servers",
"localhost:9092") .option("subscribe",
"iot_sensors") .option("startingOffsets", "earliest") .load())
```

### Gestion Automatique :

- Spark crée un consumer group
  - Gestion des offsets dans le checkpoint
  - Parallélisme : 1 Spark task par partition Kafka
  - Récupération automatique après panne
- 

## 8. Cas d'Usage SmartTech

### 8.1 Détection d'Anomalies en Temps Réel

**Pipeline** : Kafka → Spark Streaming → Delta Silver → Alertes

**Logique** : ```python anomalies =  
(silverstream .filter(col("anomalydetected") == True) .select("sensorid",  
"buildingid", "timestamp", "temperature", "humidity", "co2\_level"))

anomalies.writeStream.foreach(sendalerttoopsteam).start() ```

### Bénéfices :

- Détection en < 10 secondes
- Réduction des incidents non détectés de 80%
- Intervention préventive (maintenance prédictive)

### 8.2 Tableaux de Bord Temps Réel

**Pipeline** : Kafka → Spark Streaming → Delta Gold → Power BI

### Agrégations :

- Consommation énergétique live par bâtiment
- Température moyenne par étage
- Taux d'occupation en temps réel

**Refresh** : Toutes les 30 secondes

### 8.3 Optimisation Énergétique

**Pipeline** : Delta Silver → ML Model → Recommandations

**Analyses :**

- Corrélation occupation / consommation
- Détection de gaspillage (chauffage avec fenêtres ouvertes)
- Prédiction de la demande énergétique

**ROI SmartTech :** Réduction de 15% de la consommation énergétique

---

## 9. Bonnes Pratiques

### 9.1 Performance

- **Partitionnement** : Aligner partitions Kafka et Spark pour parallélisme optimal
- **Batch Size** : Trigger ProcessingTime adapté au volume (5-30s)
- **Watermarks** : Équilibrer entre complétude et mémoire (15-30 min)
- **Broadcast Joins** : Pour jointures avec petites tables de référence

### 9.2 Fiabilité

- **Checkpointing** : Toujours configurer checkpointLocation
- **Delta Lake** : Préférer Delta aux Parquet pour ACID
- **Monitoring** : Surveiller lag Kafka, latence de traitement
- **Idempotence** : Assurer que les sinks supportent les rejeux

### 9.3 Scalabilité

- **Partitions Kafka** : Augmenter avec le volume de données
  - **Spark Executors** : Scaler horizontalement (ajout de workers)
  - **Delta Optimize** : Lancer régulièrement OPTIMIZE et VACUUM
- 

## 10. Conclusion

### 10.1 Résumé des Concepts

| Concept             | Utilité SmartTech               | Impact               |
|---------------------|---------------------------------|----------------------|
| Streaming Structuré | Traitement unifié batch/stream  | Simplification code  |
| Checkpointing       | Reprise après panne             | Fiabilité 99.9%      |
| Modes de Sortie     | Optimisation selon use case     | Performance +50%     |
| Triggers            | Contrôle latence/throughput     | Latence < 10s        |
| Watermarks          | Gestion données tardives        | Mémoire optimisée    |
| Médailleon          | Qualité progressive des données | Traçabilité complète |
| Kafka               | Découplage et scalabilité       | Throughput +10x      |

## 10.2 Bénéfices pour SmartTech

- **Temps réel** : Détection d'anomalies en < 10 secondes
- **Fiabilité** : 99.9% de disponibilité avec checkpointing
- **Scalabilité** : Gestion de millions d'événements/jour
- **Traçabilité** : Architecture Médaillon garantit l'audit
- **ROI** : 15% d'économie d'énergie grâce aux insights temps réel

## 10.3 Évolutions Futures

- **Fenêtres de session** : Détection de séquences d'anomalies
- **ML en streaming** : Modèles de prédiction en temps réel
- **Stream-stream joins** : Corrélation multi-capteurs
- **Change Data Capture (CDC)** : Intégration avec bases de données

---

## Références

- [Spark Structured Streaming Guide](#)
- [Delta Lake Documentation](#)
- [Apache Kafka Documentation](#)
- [Databricks Medallion Architecture](#)

---

**Document rédigé par** : Équipe SmartTech

**Date** : Décembre 2025

**Version** : 1.0